



UNIVERSIDAD DE LOS LAGOS

LABORATORIO PRÁCTICO N°2 IMPLEMENTACIÓN DE STACK WEB CON DOCKER

IMPLEMENTACIÓN DE STACK WEB CON DOCKER

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
TALLER DE INGENIERÍA INFORMÁTICA
CAMPUS MEYER, OSORNO, CHILE

Gerardo Cerón
Joaquín Lespai
Tomás Teihuel

3 de junio de 2026



5 **UNIVERSIDAD ACREDITADA**
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

Índice general

1.		1
1.1.	Introducción	1
1.2.	Desarrollo	2
2.		3
2.1.	Construcción	3
2.2.	Ejecución	4
2.3.	Logs y red	5
3.		7
3.1.	Prueba funcional	7
3.2.	Configuración Docker	10
3.2.1.	Dockerfile	10
3.2.2.	Docker Compose	10
3.2.3.	Persistencia y aislamiento	10
3.3.	Conclusión	11
.1.	Anexos	12
.1.1.	Estructura del repositorio	12
.1.2.	Configuración Docker	12
.1.3.	Enlaces	12

Capítulo 1

1.1. Introducción

Este informe documenta el despliegue de una aplicación web dinámica basada en PHP y MariaDB, ejecutada mediante contenedores Docker y orquestada con Docker Compose. El objetivo del laboratorio fue implementar un stack web reproducible, capaz de separar la lógica de aplicación, la base de datos, la persistencia de información y la comunicación entre servicios.

La solución desarrollada cumple con las operaciones CRUD solicitadas: crear, leer, actualizar y eliminar registros desde una interfaz web. Para ello, se utilizó un contenedor de aplicación construido desde un `Dockerfile`, un contenedor de base de datos basado en una imagen oficial de MariaDB, un volumen persistente y una red virtual administrada por Docker Compose.

1.2. Desarrollo

La arquitectura se compone de dos servicios definidos en el archivo `docker-compose.yml`. El servicio `app` corresponde a la aplicación web PHP con Apache, mientras que el servicio `db` ejecuta MariaDB. Ambos servicios se comunican a través de la red `laboratorio_net`, y la base de datos conserva su información mediante el volumen `db_data`.

Componente	Descripción
app	Contenedor PHP 8.3 con Apache. Incluye el código de la aplicación y la extensión <code>pdo_mysql</code> para conectarse a la base de datos.
db	Contenedor MariaDB 11.4 basado en imagen oficial. Inicializa la tabla de contactos mediante <code>db/init.sql</code> .
db_data	Volumen persistente que conserva la información aunque los contenedores sean detenidos o recreados.
laboratorio_net	Red virtual tipo <i>bridge</i> que permite aislar la comunicación entre la aplicación y la base de datos.

Tabla 1.1: Componentes principales del stack Docker implementado.

Capítulo 2

2.1. Construcción

La construcción de la imagen de la aplicación se realizó con el comando:

Código 2.1: Construcción de la imagen de aplicación

```
1 | docker compose build
```

Este comando lee el Dockerfile ubicado en la carpeta app/. Durante el proceso se utiliza la imagen base php:8.3-apache, se instala la extensión pdo_mysql y se copia el archivo index.php al directorio de publicación de Apache.

```
alumno@debian:~$ docker compose build
[+] Building 4.1s (11/11) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 470B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 174B
=> [internal] load metadata for docker.io/library/php:8.3-apache
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/php:8.3-apache@sha256:239772b092f6f21965834817c8419152251263
=> => resolve docker.io/library/php:8.3-apache@sha256:239772b092f6f21965834817c8419152251263
=> [internal] load build context
=> => transferring context: 31B
=> CACHED [2/4] RUN docker-php-ext-install pdo pdo_mysql
=> CACHED [3/4] WORKDIR /var/www/html
=> CACHED [4/4] COPY index.php /var/www/html/index.php
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1be09569cfa69ff47bf30fc4e7ff2eafcff233e4619d41962055790793a9
=> => exporting config sha256:1e5e0a6b800a65a943ecfc55ea4457a5f2302f929504464a476e82a0bd4ebf
=> => exporting attestation manifest sha256:8d9104e7c0d0066d390fb562c534b9bbd7fc054b23d83ad5
=> => exporting manifest list sha256:0d7f84c5ccaa5b599253807cfbe62b13dcbfe1d41e58332a5a81563
=> => naming to docker.io/library/alumno-app:latest
=> => unpacking to docker.io/library/alumno-app:latest
=> resolving provenance for metadata file
[+] build 1/1
✔ Image alumno-app Built
```

Figura 2.1: Construcción de la imagen Docker de la aplicación.

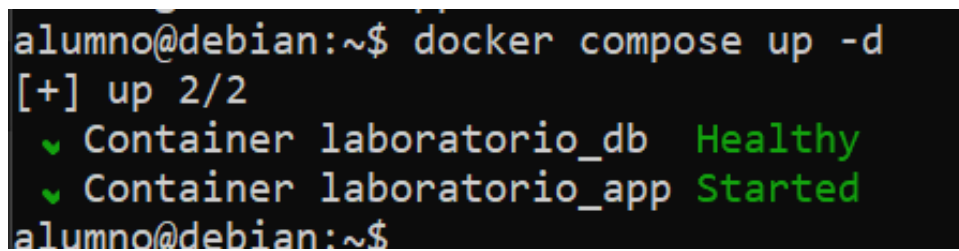
2.2. Ejecución

Una vez construida la imagen, los contenedores se levantaron en segundo plano mediante:

Código 2.2: Ejecución de los servicios

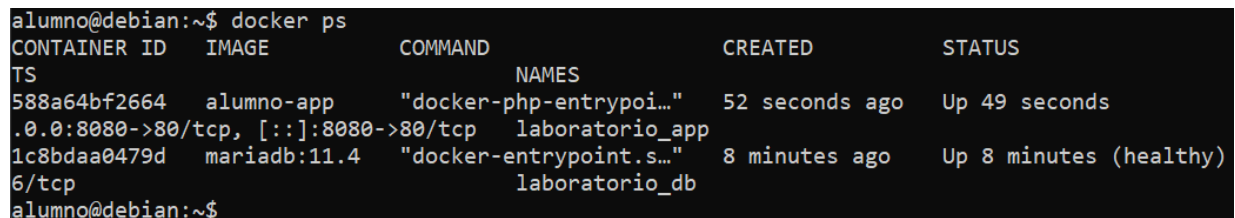
```
1 docker compose up -d
2 docker ps
```

El parámetro `-d` permite ejecutar los servicios en modo desacoplado. Posteriormente, el comando `docker ps` verifica que los contenedores `laboratorio_app` y `laboratorio_db` estén activos. En esta etapa también se confirma la publicación del puerto 8080, que redirige hacia el puerto 80 del contenedor web.



```
alumno@debian:~$ docker compose up -d
[+] up 2/2
✓ Container laboratorio_db Healthy
✓ Container laboratorio_app Started
alumno@debian:~$
```

Figura 2.2: Levantamiento de los contenedores con Docker Compose.



```
alumno@debian:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
588a64bf2664   alumno-app    "docker-php-entrypoi..." 52 seconds ago Up 49 seconds
.0.0:8080->80/tcp, [::]:8080->80/tcp   laboratorio_app
1c8bdaa0479d   mariadb:11.4  "docker-entrypoint.s..." 8 minutes ago  Up 8 minutes (healthy)
6/tcp                                                laboratorio_db
alumno@debian:~$
```

Figura 2.3: Verificación de contenedores activos mediante `docker ps`.

2.3. Logs y red

Para comprobar el inicio correcto de MariaDB, se revisaron los logs del contenedor de base de datos:

Código 2.3: Revisión de logs de la base de datos

```
1 | docker logs laboratorio_db
```

Los logs permiten verificar que el servicio de base de datos se inicializó correctamente y quedó disponible para recibir conexiones. La aplicación no necesita conectarse mediante una IP fija, ya que Docker Compose permite utilizar el nombre del servicio db como host interno dentro de la red laboratorio_net.

```
alumno@debian:~$ docker logs laboratorio_db
2026-06-03 21:50:56+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:11.4.12+maria~
ubu2404 started.
2026-06-03 21:50:57+00:00 [Warn] [Entrypoint]: /sys/fs/cgroup///memory.pressure not writable, functi
onality unavailable to MariaDB
2026-06-03 21:50:57+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-06-03 21:50:57+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:11.4.12+maria~
ubu2404 started.
2026-06-03 21:50:57+00:00 [Note] [Entrypoint]: Initializing database files
2026-06-03 21:50:57 0 [Warning] mariadbd: io_uring_queue_init() failed with EPERM: sysctl kernel.io_
uring_disabled has the value 2, or 1 and the user of the process is not a member of sysctl kernel.io_
uring_group. (see man 2 io_uring_setup).
create_uring failed: falling back to libaio
2026-06-03 21:51:00+00:00 [Note] [Entrypoint]: Database files initialized
2026-06-03 21:51:00+00:00 [Note] [Entrypoint]: Starting temporary server
2026-06-03 21:51:00+00:00 [Note] [Entrypoint]: Waiting for server startup
2026-06-03 21:51:00 0 [Note] Starting MariaDB 11.4.12-MariaDB-ubu2404 source revision b4a80422bfec9
3079a430c080fffbda8f6fa574 server_uid ZN0pt2WR8CbEgr7vYHf2m/ArLuY= as process 86
2026-06-03 21:51:00 0 [Note] InnoDB: Compressed tables use zlib 1.3
2026-06-03 21:51:00 0 [Note] InnoDB: Number of transaction pools: 1
2026-06-03 21:51:00 0 [Note] InnoDB: Using crc32 + pclmulqdq instructions
2026-06-03 21:51:00 0 [Warning] mariadbd: io_uring_queue_init() failed with EPERM: sysctl kernel.io_
uring_disabled has the value 2, or 1 and the user of the process is not a member of sysctl kernel.io_
uring_group. (see man 2 io_uring_setup).
create_uring failed: falling back to libaio
2026-06-03 21:51:00 0 [Note] InnoDB: Using Linux native AIO
2026-06-03 21:51:00 0 [Note] InnoDB: innodb_buffer_pool_size_max=8388608m, innodb_buffer_pool_size=1
28m
2026-06-03 21:51:00 0 [Note] InnoDB: Completed initialization of buffer pool
2026-06-03 21:51:00 0 [Note] InnoDB: File system buffers for log disabled (block size=512 bytes)
2026-06-03 21:51:00 0 [Note] InnoDB: End of log at LSN=45568
2026-06-03 21:51:00 0 [Note] InnoDB: Opened 3 undo tablespaces
2026-06-03 21:51:00 0 [Note] InnoDB: 128 rollback segments in 3 undo tablespaces are active.
2026-06-03 21:51:00 0 [Note] InnoDB: Setting file './ibtmp1' size to 12.000MiB. Physically writing t
he file full; Please wait ...
2026-06-03 21:51:00 0 [Note] InnoDB: File './ibtmp1' size is now 12.000MiB.
2026-06-03 21:51:00 0 [Note] InnoDB: log sequence number 45568; transaction id 14
2026-06-03 21:51:00 0 [Note] Plugin 'FEEDBACK' is disabled.
2026-06-03 21:51:00 0 [Note] Plugin 'wsrep-provider' is disabled.
2026-06-03 21:51:01 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
2026-06-03 21:51:01 0 [Note] mariadbd: ready for connections.
Version: '11.4.12-MariaDB-ubu2404' socket: '/run/mysqld/mysqld.sock' port: 0 mariadb.org binary d
istribution
```

Figura 2.4: Primera parte de los logs del contenedor MariaDB.

```
2026-06-03 21:51:03 0 [Note] InnoDB: FTS optimize thread exiting.
2026-06-03 21:51:03 0 [Note] InnoDB: Starting shutdown...
2026-06-03 21:51:03 0 [Note] InnoDB: Dumping buffer pool(s) to /var/lib/mysql/ib_buffer_pool
2026-06-03 21:51:03 0 [Note] InnoDB: Buffer pool(s) dump completed at 260603 21:51:03
2026-06-03 21:51:03 0 [Note] InnoDB: Removed temporary tablespace data file: "/ibtmp1"
2026-06-03 21:51:03 0 [Note] Shutdown completed; log sequence number 49441; transaction id 23
2026-06-03 21:51:03 0 [Note] mariadbd: Shutdown complete
2026-06-03 21:51:03+00:00 [Note] [Entrypoint]: Temporary server stopped

2026-06-03 21:51:03+00:00 [Note] [Entrypoint]: MariaDB init process done. Ready for start up.

2026-06-03 21:51:03 0 [Note] Starting MariaDB 11.4.12-MariaDB-ubu2404 source revision b4a80422bfeec9
3079a430c080ffffbda8f6fa574 server_uid ZN0pt2WR8CbEgr7vYHf2m/ArluYz as process 1
2026-06-03 21:51:03 0 [Note] InnoDB: Compressed tables use zlib 1.3
2026-06-03 21:51:03 0 [Note] InnoDB: Number of transaction pools: 1
2026-06-03 21:51:03 0 [Note] InnoDB: Using crc32 + pclmulqdq instructions
2026-06-03 21:51:03 0 [Warning] mariadbd: io_uring_queue_init() failed with EPERM: sysctl kernel.io_
uring_disabled has the value 2, or 1 and the user of the process is not a member of sysctl kernel.io
uring_group. (see man 2 io_uring_setup).
create_uring failed: falling back to libaio
2026-06-03 21:51:03 0 [Note] InnoDB: Using Linux native AIO
2026-06-03 21:51:03 0 [Note] InnoDB: innodb_buffer_pool_size_max=8388608m, innodb_buffer_pool_size=1
28m
2026-06-03 21:51:03 0 [Note] InnoDB: Completed initialization of buffer pool
2026-06-03 21:51:03 0 [Note] InnoDB: File system buffers for log disabled (block size=512 bytes)
2026-06-03 21:51:03 0 [Note] InnoDB: End of log at LSN=49441
2026-06-03 21:51:03 0 [Note] InnoDB: Opened 3 undo tablespaces
2026-06-03 21:51:03 0 [Note] InnoDB: 128 rollback segments in 3 undo tablespaces are active.
2026-06-03 21:51:03 0 [Note] InnoDB: Setting file './ibtmp1' size to 12.000MiB. Physically writing t
he file full; Please wait ...
2026-06-03 21:51:03 0 [Note] InnoDB: File './ibtmp1' size is now 12.000MiB.
2026-06-03 21:51:03 0 [Note] InnoDB: log sequence number 49441; transaction id 24
2026-06-03 21:51:03 0 [Note] Plugin 'FEEDBACK' is disabled.
2026-06-03 21:51:03 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buffer_pool
2026-06-03 21:51:03 0 [Note] Plugin 'wsrep-provider' is disabled.
2026-06-03 21:51:03 0 [Note] InnoDB: Buffer pool(s) load completed at 260603 21:51:03
2026-06-03 21:51:03 0 [Note] Server socket created on IP: '0.0.0.0', port: '3306'.
2026-06-03 21:51:03 0 [Note] Server socket created on IP: '::', port: '3306'.
2026-06-03 21:51:03 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
2026-06-03 21:51:03 0 [Note] mariadbd: ready for connections.
Version: '11.4.12-MariaDB-ubu2404' socket: '/run/mysqld/mysqld.sock' port: 3306 mariadb.org binar
y distribution
```

Figura 2.5: Segunda parte de los logs del contenedor MariaDB.

Capítulo 3

3.1. Prueba funcional

La aplicación fue probada desde el navegador a través de la dirección:

Código 3.1: URL de acceso a la aplicación

```
1 | http://localhost:8080
```

En la máquina virtual Debian también fue posible acceder desde el equipo anfitrión utilizando la IP de la VM y el puerto publicado. La interfaz presenta un menú principal y una tabla de registros, permitiendo demostrar las operaciones de creación, lectura, actualización y eliminación.

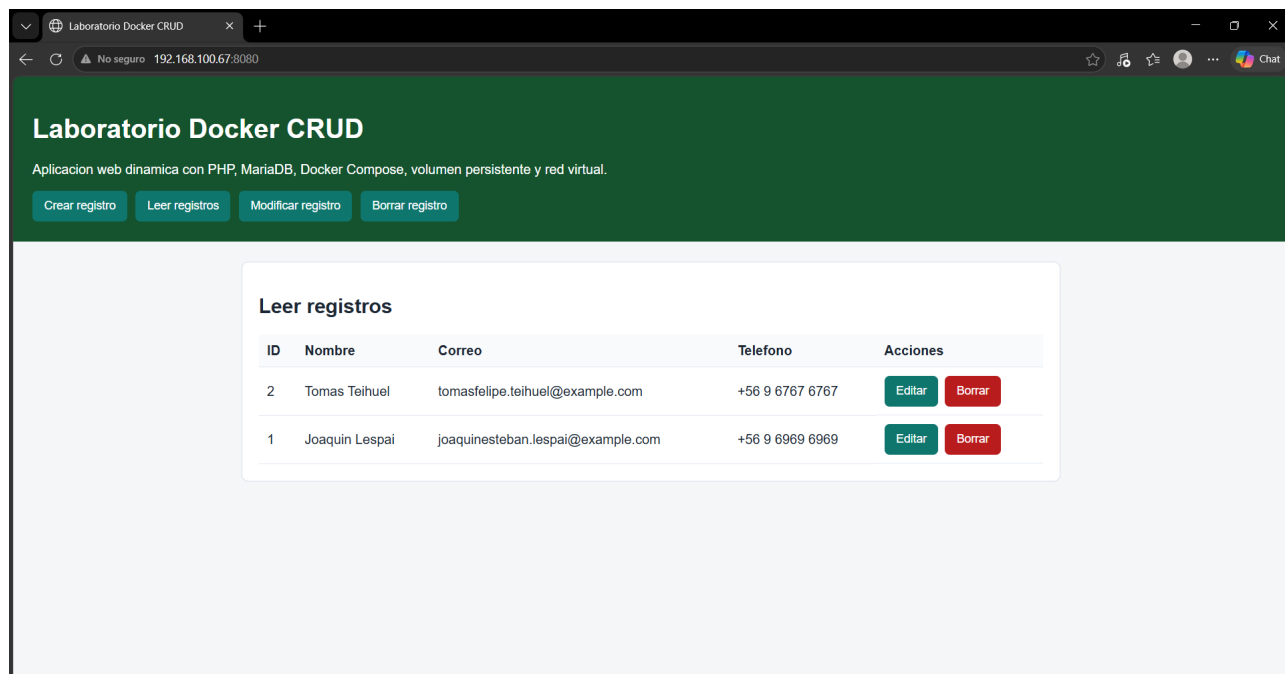


Figura 3.1: Pantalla principal de la aplicación CRUD.

Crear registro

Nombre

Correo

Telefono

Guardar

Cancelar

Figura 3.2: Formulario para crear un nuevo registro.

Modificar registro

Nombre

Tomas Teihuel

Correo

tomasfelipe.teihuel@example.com

Telefono

+56 9 6767 6767

Actualizar

Cancelar

Figura 3.3: Modificación de un registro existente.

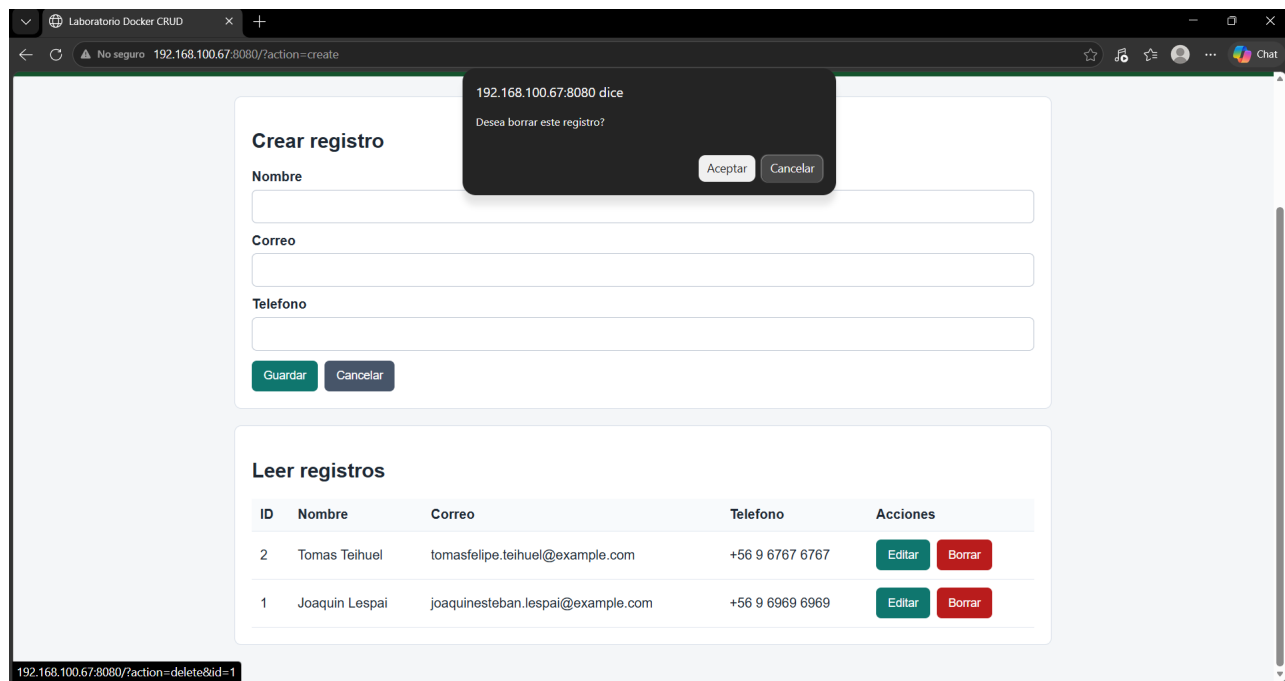


Figura 3.4: Eliminación de un registro desde la interfaz web.

3.2. Configuración Docker

3.2.1. Dockerfile

El archivo `app/Dockerfile` define la imagen de la aplicación. Se utilizó una imagen oficial de PHP con Apache, a la cual se le agregó la extensión `pdo_mysql`. Esta extensión permite que la aplicación PHP se conecte a MariaDB usando PDO.

Código 3.2: Dockerfile de la aplicación

```
1 FROM php:8.3-apache
2
3 RUN docker-php-ext-install pdo pdo_mysql
4
5 WORKDIR /var/www/html
6 COPY index.php /var/www/html/index.php
7
8 EXPOSE 80
```

3.2.2. Docker Compose

El archivo `docker-compose.yml` vincula los servicios `app` y `db`. La aplicación expone el puerto 8080 hacia el navegador, mientras que la base de datos se mantiene dentro de la red interna y utiliza variables de entorno para definir usuario, contraseña y nombre de la base de datos.

Código 3.3: Comandos principales de despliegue

```
1 cp .env.example .env
2 docker compose build
3 docker compose up -d
4 docker ps
5 docker logs laboratorio_db
```

3.2.3. Persistencia y aislamiento

La persistencia se implementó mediante el volumen `db_data`, montado sobre `/var/lib/mysql`. Gracias a esto, los datos creados desde la aplicación se conservan incluso si los contenedores son detenidos o recreados. El aislamiento se logró mediante la red virtual `laboratorio_net`, evitando exponer innecesariamente el servicio de base de datos fuera del entorno Docker.

3.3. Conclusión

La implementación del laboratorio permitió integrar una aplicación web dinámica en PHP con una base de datos MariaDB desplegada mediante Docker, cumpliendo con los requisitos de CRUD, persistencia y aislamiento de servicios. El uso de `docker compose` simplificó la orquestación del entorno y facilitó reproducir el sistema de manera consistente en distintos equipos.

Durante el desarrollo se comprobó que la separación entre el contenedor de aplicación y el contenedor de base de datos mejora el mantenimiento de la solución, porque cada servicio cumple una responsabilidad específica y se comunica a través de una red virtual controlada. Además, el uso de volúmenes permitió conservar la información aun cuando los contenedores fueron reiniciados o recreados.

En términos funcionales, la interfaz web entrega un flujo claro para crear, leer, actualizar y eliminar registros, lo que valida el objetivo principal del laboratorio.

.1. Anexos

.1.1. Estructura del repositorio

Código 4: Estructura de archivos del proyecto

```
1 app/  
2   index.php  
3   Dockerfile  
4 db/  
5   init.sql  
6 informe/  
7   proyecto.tex  
8   proyecto.pdf  
9 .env.example  
10 .gitignore  
11 docker-compose.yml  
12 README.md
```

.1.2. Configuración Docker

- app/Dockerfile: imagen de la aplicación PHP con Apache.
- docker-compose.yml: orquestación de servicios, red, volumen y variables de entorno.
- db/init.sql: script de inicialización de la base de datos.
- .env.example: ejemplo de variables de entorno necesarias para ejecutar el proyecto.

.1.3. Enlaces

- Repositorio GitHub: <https://github.com/TomasTeihuel/lab-docker-crud>